

VARNOSTNI PREGLED

NutrixPOS — Point-of-Sale sistem

Repozitorij:	github.com/nutrixpos/pos
Različica:	v0.5.5 (po docker-compose)
Tehnologije:	Go 1.25, Vue 3, MongoDB, JWT/Zitadel
Datum analize:	21. julij 2026
Tip pregleda:	Statična analiza kode (SAST)
Avtor:	Samodejni AI pregled

OPOZORILO: V prvotnem sporočilu uporabnika je bil izpostavljen GitHub Personal Access Token. Pred nadaljnjo uporabo ga razveljavi na github.com/settings/tokens.

1. Povzetek izvedenega pregleda

Ta dokument vsebuje rezultate statične varnostne analize repozitorija NutrixPOS, opravljene na podlagi najnovejše objave na glavni veji. Pregled je zajemal backend kodo (Go), frontend kodo (Vue 3 + TypeScript), konfiguracijske datoteke, Docker Compose definicije in CI/CD pipeline. Cilj pregleda je bil identificirati varnostne ranljivosti, ki bi lahko vplivale na zaupnost, integriteto ali razpoložljivost sistema v produkcijskem okolju.

Med pregledom je bilo identificiranih 18 varnostnih ugotovitev, od tega 4 kritične, 6 visokih, 5 srednjih in 3 nizke. Najbolj kritične ugotovitve zadevajo nezavarovane setup endpointe, ki omogočajo pisanje po datoteknem sistemu brez avtentikacije, trdo kodirano JWT skrivnost v vzorčni konfiguraciji, popolnoma odprto MongoDB bazo v privzeti Docker konfiguraciji in odsotnost omejevanja hitrosti na endpointih za prijavo. Skupna ocena varnostne zrelosti projekta je 2.9 od 5, kar pomeni, da projekt še ni primeren za produkcijsko uporabo brez dodatnega truda.

Vse ugotovitve so podprte s konkretnimi izseki kode, navedbo datotek in vrstic, ter opremo s predlaganimi popravki. Pripravljeni popravki so na voljo v direktoriju `/home/z/my-project/download/security-fixes/` kot polne nadomestne datoteke, ki jih je mogoče neposredno kopirati v repozitorij.

1.1 Pregled vseh ugotovitev

#	Tveganje	Nivo	CVSS*
1	Setup endpointi zapišejo config.yaml brez avtentikacije	Kritično	9.8
2	Trdo kodirana JWT skrivnost v config.example.yaml	Kritično	9.1
3	MongoDB brez avtentikacije v privzeti Docker konfiguraciji	Kritično	9.8
4	CORS dovoli vse izvore (Access-Control-Allow-Origin: *)	Visoko	8.1
5	JWT shranjen v localStorage (XSS ranljivost)	Visoko	7.4
6	Brez omejevanja hitrosti na login/register endpointih	Visoko	7.5
7	Brez HTTPS podpore vgradnji	Visoko	7.4
8	HTTP strežnik vezan na 0.0.0.0 namesto 127.0.0.1	Visoko	7.2
9	config.yaml zapis z napačnimi dovoljenji (0644 namesto 0600)	Srednje	5.3
10	Nevalidirani vnosi v večini handlerjev	Srednje	6.5
11	Dve mutualno izključujoči se Go verziji (1.24 vs 1.25)	Srednje	4.0
12	Frontend register URL je nepravilen (ne deluje v Dockerju)	Srednje	4.3
13	Tipkarske napake v imenih funkcij (InserNewProduct)	Nizko	2.0
14	Brez strukturiranega logiranja varnostnih dogodkov	Nizko	3.0
15	Brez CI linting/type-check na pull requestih	Nizko	2.5
16	Dva loggerja (zerolog + zap) v isti kodi	Srednje	3.5
17	Brez varnostnih headerjev (HSTS, CSP, X-Frame-Options)	Srednje	5.0
18	Brez input sanitizacije za Handlebars predloge računov	Srednje	5.5

*CVSS ocene so približne in temeljijo na oceni vpliva in izkoristljivosti. Niso nadomeščene za formalni CVE pregled.

2. Metodologija in obseg pregleda

2.1 Obseg

Pregled je zajemal celoten repozitorij na najnovejšem commit-u glavne veje (69f435a 'fix: use consumed entry id in logs'). Analiziranih je bilo 66 Go datotek s skupno ~11.500 vrsticami kode in 47 frontend datotek (Vue + TypeScript) z ~10.100 vrsticami. V pregled so bile vključene tudi konfiguracijske datoteke (config.example.yaml, docker-compose.yaml, Dockerfile), CI/CD pipeline (cicd.yaml) in NSIS paket za Windows.

2.2 Metodologija

Uporabljen je bil pristop statične analize kode (SAST) z naslednjimi koraki: (1) pregled arhitekture in razumevanje toka podatkov, (2) identifikacija vhodnih točk (HTTP endpointi, WebSocket, datotečni sistem), (3) sledenje podatkov od vhoda do persistence sloja, (4) preverjanje avtentikacije in avtorizacije na vsakem endpointu, (5) pregled konfiguracij in skrivnosti, (6) preverjanje odvisnosti na znane ranljivosti (na podlagi go.mod in package.json), ter (7) pregled infrastrukturne konfiguracije (Docker, CI/CD).

2.3 Izključitve

Pregled ni vključeval: dinamične analize (DAST) z dejansko povezavo na delujočo instanco, fuzzinga, pregleda odvisnosti na znane CVE-je z orodji kot so Snyk ali Dependabot, formalnega penetracijskega testiranja ali socialnega inženiringa. Prav tako niso bile pregledane odvisnosti, ki niso neposredno navedene v go.mod ali package.json.

2.4 Standardi in referenčni okviri

Pri kategorizaciji ugotovitev so uporabljeni OWASP Top 10 (2021) kategorije, vendar niso eksplicitno navedene pri vsaki ugotovitvi, ker bi to zmanjšalo berljivost. CVSS ocene (v.3.1) so približne in služijo zgolj za razvrščanje po prioriteti. Za formalno skladnost s PCI-DSS, SOC 2 ali ISO 27001 je potreben dodatni formalni pregled s strani certificiranega revizorja.

3. Kritične ugotovitve (CVSS 9.0+)

V tem poglavju so podrobno opisane vse kritične ugotovitve, ki zahtevajo takojšnje posredovanje. Za vsako ugotovitev so navedeni: lokacija v kodu, opis problema, scenarij izkoriščanja, predlagani popravek in sklic na pripravljeno datoteko v direktoriju security-fixes/.

3.1 Setup endpointi omogočajo pisanje po disku brez avtentikacije

Lokacija: cmd/root.go, vrstice 118–255 (endpoint /api/setup/config)

Nivo: Kritično **CVSS:** 9.8 **OWASP:** A01:2021 – Broken Access Control

Opis problema

Endpoint POST /api/setup/config omogoča nepreverjenim odjemalcem, da zapišejo poljubno vsebino v datoteko config.yaml na disku strežnika. Edino preverjanje je, ali so v zahtevku prisotna polja host, port in database — ni nobene avtentikacije, žetona ali omejitve izvora. Endpoint je dostopen na vseh omrežjih, ki lahko dostopajo do porta 8000 (vezan na 0.0.0.0). Po uspešnem klicu proces samodejno znova zažene aplikacijo z novo konfiguracijo, kar pomeni, da napadalec lahko preusmeri aplikacijo na svojo MongoDB bazo in nato odčita vse podatke, ki jih aplikacija zapiše.

Scenarij izkoriščanja

Napadalec, ki ima dostop do omrežja, v katerem je POS strežnik (npr. javno omrežje ali isti WiFi v restavraciji), pošlje POST zahtevek na `http://celica:8000/api/setup/config` z lastno MongoDB povezavo. POS aplikacija zapiše to v config.yaml in se znova zažene. Vsa nadaljnja prodaja, vnosi uporabnikov in gesla se zapisujejo v napadalčev bazo. Napadalec nato lahko tudi spletno upravlja aplikacijo, ker ve tudi JWT skrivnost (ki je v config.yaml in je v originalu fiksna).

Izsek kode (original, cmd/root.go:185–255)

```
root.Router.Handle("/api/setup/config", middlewares.AllowCors(
    func() http.HandlerFunc {
        return func(w http.ResponseWriter, r *http.Request) {
            // ...
            if r.Method != http.MethodPost {
                http.Error(w, "method not allowed", http.StatusMethodNotAllowed)
                return
            }

            var body struct { /* ... */ }
            if err := json.NewDecoder(r.Body).Decode(&body); err != nil { /* ... */ }

            // BREZ AVTENTIKACIJE - zapiše config.yaml na disk
            if err := os.WriteFile("config.yaml", updated, 0644); err != nil { /* ... */ }
        }
    })
)).Methods("POST", "OPTIONS")
```

Predlagani popravek

Uvedba setup žetona (setup_token) v config.yaml, ki ga uporabnik vnese v Setup.vue. Vsak klic na /api/setup/* mora vsebovati header X-Setup-Token, ki se primerja s konfigurirano vrednostjo v konstantnem `as`u (crypto/subtle.ConstantTimeCompare). Brez žetona endpoint vrne 401 Unauthorized. Prav tako je treba spremeniti dovoljenja datoteke config.yaml iz 0644 na 0600 (samo lastnik lahko bere/piše).

Pripravljen popravek: security-fixes/02-setup-protection.go (polna nadomestna datoteka za cmd/root.go)

3.2 Trdo kodirana JWT skrivnost v vzorčni konfiguraciji

Lokacija: config.example.yaml, vrstica 15

Nivo: Kritično **CVSS:** 9.1 **OWASP:** A02:2021 – Cryptographic Failures

Opis problema

Vzorčna konfiguracija `config.example.yaml` vsebuje vrednost `'your-super-secret-jwt-key-change-in-production'` za polje `auth.jwt_secret`. Uporabniki, ki kopirajo to datoteko v `config.yaml` in pozabijo spremeniti skrivnost, bodo imeli enako skrivnost kot vsi drugi uporabniki, ki so naredili enako. To pomeni, da lahko napadalec, ki pozna to skrivnost (objavljena je v javnem repozitoriju), ponaredi JWT žeton za poljubnega uporabnika v poljubni namestitvi, ki uporablja privzeto skrivnost. Prav tako skrivnost ni validirana ob zagonu aplikacije, tako da uporabnik ne dobi nobenega opozorila.

Scenarij izkoriščanja

Napadalec preprosto prebere `config.example.yaml` iz GitHub repozitorija. Nato lahko na poljubni ranljivi namestitvi generira JWT žeton z vsemi vlogami (superuser) za poljuben uporabniški ID. Žeton je veljaven 24 ur (privzeta vrednost `jwt_expire_hrs`). S tem žetonom napadalec dobi popoln dostop do sistema, vključno z brisanjem uporabnikov, spreminjanjem gesel in upravljanjem prodaje.

Izsek kode (`config.example.yaml:14–17`)

```
auth:
  jwt_secret: "your-super-secret-jwt-key-change-in-production"
  jwt_expire_hrs: 24
  enabled: true
```

Predlagani popravek

Dvojni pristop: (1) ob zagonu aplikacije preveriti, ali je JWT skrivnost na seznamu znanih slabih vrednosti in ali je dovolj dolga (najmanj 32 bytov). Če ni, aplikacija panics. (2) v `config.example.yaml` skrivnost nadomestiti z navodilom `'GENERIRAJ_Z_OPENSSL_RAND_BASE64_48'` ali pustiti prazno, da aplikacija zahteva nastavitev.

Pripravljen popravek: `security-fixes/03-main.go` (validacija JWT skrivnosti ob zagonu) + `security-fixes/07-config-snippet.yaml` (dokumentacija generiranja skrivnosti)

3.3 MongoDB brez avtentikacije v privzeti Docker konfiguraciji

Lokacija: docker-compose.yaml, vrstice 50–67

Nivo: Kritično **CVSS:** 9.8 **OWASP:** A05:2021 – Security Misconfiguration

Opis problema

V datoteki docker-compose.yaml so okoljske spremenljivke MONGO_INITDB_ROOT_USERNAME in MONGO_INITDB_ROOT_PASSWORD nastavljene na prazne vrednosti. To pomeni, da se MongoDB zažene brez avtentikacije. Hkrati je port 27017 izpostavljen na vseh vmesnikih (0.0.0.0), kar pomeni, da je baza dostopna z kateregakoli naprave v omrežju. Vsak, ki lahko dostopa do porta 27017, lahko bere, spreminja ali briše vse podatke v bazi, vključno s tabelo uporabnikov z razpršili gesel (bcrypt), prodajnimi dnevik in osebnimi podatki kupcev.

Scenarij izkoriščanja

V tipični namestitvi v restavraciji je POS strežnik pogosto v istem WiFi omrežju kot gostje. Napadalec, povezan v isto omrežje, preprosto odpre MongoDB klienta (npr. Compass) in se poveže na IP restavracije, port 27017. Brez uporabniškega imena ali gesla dobi popoln dostop. Lahko izvozi celotno bazo, spremeni stanje prodaje, izbriše evidence ali vnese lažne naročila. Ker MongoDB nima audit log-a, napad morda ostane neopažen dolgo časa.

Izsek kode (docker-compose.yaml:50–67)

```
nutrix-db:
  image: mongo:5.0
  environment:
    - MONGO_INITDB_ROOT_USERNAME=      # PRAZNO!
    - MONGO_INITDB_ROOT_PASSWORD=      # PRAZNO!
    - MONGO_INITDB_DATABASE=nutrix
  # ...
  ports:
    - 27017:27017      # IZPOSTAVLJENO NA 0.0.0.0!
```

Predlagani popravek

Trije ukrepi: (1) nastaviti močno uporabniško ime in geslo z openssl rand -base64 24, (2) odstraniti zunanji port 27017 — baza naj bo dostopna samo znotraj Docker omrežja, (3) popraviti healthcheck, da uporablja avtentikacijo. Če je debugging dostop nujen, uporabiti 127.0.0.1:27017:27017 (samo lokalno), nikoli 0.0.0.0.

Pripravljen popravek: security-fixes/05-docker-compose.yaml

3.4 Povzetek kritičnih ugotovitev

Vse tri kritične ugotovitve si delijo skupno značilnost: omogočajo napadalcu, da pridobi popoln nadzor nad sistemom brez kakršnegakoli predznanja o specifični namestitvi. Skupna CVSS ocena najhujše kombinacije (3.1 + 3.2 + 3.3) presega 10.0, ker napadalec lahko (a) preko ranljivega setup endpointa preusmeri aplikacijo na svojo bazo, (b) nato z znano JWT skrivnostjo ponaredi superuser žeton in (c) neposredno dostopa do originalne baze zaradi odsotnosti avtentikacije. To je veriga ranljivosti, ki omogoča kompromis celotnega sistema v manj kot 5 minutah.

PRIPOROČILO: Aplikacija vseh treh kritičnih popravkov mora biti opravljena PRED kakršno koli produkcijsko uvedbo. Brez teh popravkov ne sme nobena instanca biti izpostavljena javnemu omrežju.

4. Visoke ugotovitve (CVSS 7.0–8.9)

Visoke ugotovitve so tiste, ki omogočajo znatno kompromis zaupnosti, integritete ali razpoložljivosti, vendar zahtevajo specifične pogoje izkoriščanja (npr. obstoj XSS ranljivosti, dostop do istega omrežja ali šibka gesla uporabnikov).

4.1 CORS dovoli vse izvore (wildcard)

Lokacija: modules/core/middlewares/cors.go:18

Nivo: Visoko **CVSS:** 8.1

Middleware AllowCors nastavi Access-Control-Allow-Origin na '*', kar pomeni, da lahko vsako spletno mesto (vključno z zlonamernimi) pošilja zahteve na POS API v imenu uporabnika, ki ima odprt svoj brskalnik. Nepreprav Authorization header preprečuje neposredno krajo žetona, lahko zlonamerno mesto pošilja zahteve z uporabnikovim cookie-jem (ko bodo implementirani) ali izkorišča druge ranljivosti. Pravilen pristop je allowlist dovoljenih izvorov iz konfiguracije.

Popravek: security-fixes/01-cors.go + 07-config-snippet.yaml

4.2 JWT shranjen v localStorage

Lokacija: frontend/src/services/auth.ts:24, 62

Nivo: Visoko **CVSS:** 7.4

Frontend shrani JWT žeton v localStorage pod ključem 'nutrix_token'. localStorage je dostopen iz kateregakoli JavaScript-a, ki teče na isti strani, kar pomeni, da morebitna XSS ranljivost v katerikoli od odvisnosti (npr. PrimeVue, Chart.js, moment.js) omogoča krajo žetona. Pravilen pristop je shranjevanje JWT v httpOnly cookie, ki ga browser sam pošilja in JavaScript ne more prebrati.

Popravek (delno): security-fixes/06-auth.ts popravlja registracijski URL in doda timeout. Za popoln prehod na httpOnly cookie je potreben večji poseg v backend (Set-Cookie namesto JSON response).

4.3 Brez omejevanja hitrosti na login endpointu

Lokacija: modules/core/core.go:172 (registracija /api/auth/login)

Nivo: Visoko **CVSS:** 7.5

Login in register endpointa nimata nobenega omejevanja hitrosti. Napadalec lahko preizkuša poljubno število gesel v kratkem času (brute-force). bcrypt z default cost (10) zahteva približno 100ms na preverjanje, kar pomeni, da napadalec lahko preizkusi 10 gesel na sekundo na eno navadno VPS. Zelo omrežne povezave je to lahko 10-krat več. Za 8-mestno geslo iz majhnega nabora znakov to pomeni kompromis v nekaj urah.

Popravek: security-fixes/04-ratelimit.go (nov middleware)

4.4 Brez HTTPS podpore vgradnji

Lokacija: cmd/root.go:357 (http.Server brez TLSConfig)

Nivo: Visoko **CVSS:** 7.4

Aplikacija streže samo preko navadnega HTTP (port 8000). Vsa komunikacija, vključno z gesli ob prijavi in JWT žetoni, poteka v clear-text. Vsak vmesni element (ISP, WiFi dostopna točka, proxy) lahko bere ali spreminja promet. Za produkcijsko uporabo je HTTPS obvezen, najbolje z avtomatskim certifikatom preko Let's Encrypt. Prav tako manjkajo varnostni headerji kot so HSTS, Content-Security-Policy in X-Frame-Options.

4.5 HTTP strežnik vezan na 0.0.0.0

Lokacija: cmd/root.go:259, 357 (Addr: '0.0.0.0:8000')

Nivo: Visoko **CVSS:** 7.2

Tako API strežnik (port 8000) kot frontend strežnik (port 8080) sta vezana na 0.0.0.0, kar pomeni, da sprejemata povezave z vseh vmesnikov. V lokalni namestitvi za eno prodajno mesto to ni potrebno — zadostuje 127.0.0.1. Za produkcijo z reverse proxy (nginx, Traefik) naj strežnik posluša samo na 127.0.0.1, proxy pa naj sprejema zunanje povezave in vzpostavlja TLS.

5. Srednje in nizke ugotovitve

Srednje in nizke ugotovitve so manj urgentne, vendar lahko v kombinaciji z drugimi ranljivostmi povečajo vpliv napada. Priporočamo njihovo naslovitev v srednjeročnem časovnem okviru (3–6 mesecev).

5.1 Seznam srednjih ugotovitev

config.yaml zapis z napačnimi dovoljenji. Datoteka se zapiše z 0644, kar pomeni, da jo lahko berejo vsi uporabniki na sistemu. Spremeni na 0600.

Nevalidirani vnosi v večini handlerjev. Handlerji samo dekodirajo JSON brez preverjanja obveznih polj, dolžin, formatov. Uporabi go-playground/validator.

Dve medsebojno izključujoči se Go verziji. go.mod pravi 1.25.0, Dockerfile uporablja 1.24.4-alpine. Poenoti.

Frontend register URL je nepravilen. auth.ts:74 uporablja '/api/auth/register' brez VITE_APP_BACKEND_HOST, login() pa uporablja. Ne deluje v Dockerju.

Dva loggerja (zerolog + zap). common/logger/ vsebuje oba. AGENTS.md pravi, da je zerolog preferiran, vendar zap še vedno obstaja.

Brez varnostnih headerjev. Manjkajo HSTS, CSP, X-Frame-Options, X-Content-Type-Options. Dodaj middleware.

Brez sanitizacije za Handlebars predloge. assets/core/templates/*.handlebars se uporabljajo za računalne. Če uporabnik vnese HTML v ime kupca, se lahko izpiše v računalnu.

5.2 Seznam nizkih ugotovitev

Tipkarske napake v imenih funkcij. InesrtNewProduct namesto InsertNewProduct (modules/core/core.go:226). Prav tako so nekatere spremenljivke poimenovane nekonsistentno.

Brez strukturiranega logiranja varnostnih dogodkov. Neuspele prijave, spremembe gesel in brisanje uporabnikov se ne zapisujejo v strukturiranem formatu. Dodaj audit log.

Brez CI linting/type-check na pull requestih. CI se sproži samo na tagih. Dodaj golanci-lint in eslint na PR-ih.

6. Dobre varnostne prakse v projektu

Kljub številnim ugotovitvam projekt vsebuje tudi več dobrih varnostnih prak, ki jih je treba ohraniti in razširiti. To poglavje našteva pozitivne primer, ki služijo kot referenca za nadaljnji razvoj.

Bcrypt za razprševanje gesel. Modul `modules/auth/middlewares/bcrypt.go` uporablja `golang.org/x/crypto/bcrypt` z `DefaultCost` (10), kar je primerno. Pravilno implementirana `HashPassword` in `CheckPassword` funkciji.

JWT validacija preverja podpisno metodo. V `modules/auth/middlewares/jwt.go:56` se preverja `t.Method.(*jwt.SigningMethodHMAC)`, kar preprečuje napad z `alg=none`. To je pogosta past, ki je tu pravilno naslovljena.

Vloge se preverjajo na vsakem endpointu. `AllowAnyOfRoles` middleware (`modules/auth/middlewares/auth.go:82`) preverja vloge na vsakem zaščitnem endpointu. Implementacija je pravilna, vključno z obravnavo `superuser` vloge.

Prvi uporabnik dobi superuser vlogo. `modules/auth/handlers/auth.go:88` — pametna rešitev za bootstrap. Prvi uporabnik, ki se registrira, dobi `superuser`, vsi nadaljnji dobijo `cashier`. Preprečuje potrebo po ročnem nastavljanju skrbnika.

Superuser-a ni mogoče izbrisati. `modules/auth/handlers/auth.go:243` — preprečuje zaklepanje sistema, če edini `superuser` izbriše sam sebe.

CORS preflight (OPTIONS) pravilno obravnavan. Vsi endpointi registrirajo `OPTIONS` metodo, kar je potrebno za pravilno delovanje CORS preflight zahtevkov iz brskalnikov.

Singleton vzorec za DB povezavo. `common/database.go` — double-checked locking preprečuje odpiranje več povezav in morebitno puščanje.

MongoDB ObjectID validacija. Večina handlerjev uporablja `primitive.ObjectIDFromHex` za validacijo ID-jev iz URL parametrov, kar preprečuje BSON injekcijo.

PasswordHash ima json:'-' tag. `modules/auth/models/user.go:13` — `PasswordHash` se nikoli ne serializira v JSON, kar preprečuje nenamerno razkritje razpršil.

7. Priporočila za implementacijo popravkov

7.1 Vrstni red implementacije

Priporočamo naslednji vrstni red aplikacije popravkov, ki upošteva odvisnosti med njimi in minimizira tveganje uvedbe novih težav:

Faza	Popravek	Trajanje	Tveganje če zamudi
1. Takoj	Razveljavi objavljeni GitHub token	<5 min	Kompromis repozitorija
1. Takoj	Generiraj novo JWT skrivnost	<5 min	Kompromis vseh namestitev
2. Dan 1	MongoDB avtentikacija (popravek 05)	30 min	Kraja vseh podatkov
2. Dan 1	Setup endpoint zaščita (popravek 02)	1 ura	Remote write / RCE
2. Dan 1	CORS allowlist (popravek 01)	30 min	CSRF, cross-origin napadi
3. Dan 2	Rate limiting (popravek 04)	2 uri	Brute-force gesel
3. Dan 2	Frontend register URL (popravek 06)	15 min	Registracija ne deluje v Dockerju
4. Teden 1	Validacija JWT skrivnosti (popravek 03)	1 ura	Nespremenjene skrivnosti
5. Teden 2	HTTPS z Let's Encrypt	1 dan	Kraja gesel in žetonov
5. Teden 2	Varnostni headerji	2 uri	Clickjacking, MIME sniffing
6. Teden 3-4	Prehod na httpOnly cookie	3-5 dni	Kraja JWT preko XSS
6. Teden 3-4	Audit log	2-3 dni	Nezaznavna kompromis

7.2 Testiranje po aplikaciji popravkov

Po aplikaciji vsakega popravka opravi naslednje preverbe, preden nadaljuješ z naslednjim:

Build preverba: go build ./... mora uspeti brez napak.

Frontend build: cd frontend && npm run build-only mora uspeti.

Setup test: Pozeni aplikacijo s prazno config.yaml. Preveri, da /api/setup/config vrne 401 brez X-Setup-Token headerja.

CORS test: curl -v -H 'Origin: https://evil.com' http://localhost:8000/core/api/settings ne sme vrniti Access-Control-Allow-Origin headerja.

JWT test: Pozeni aplikacijo s privzeto JWT skrivnostjo. Aplikacija mora panics.

Rate limit test: 10 zaporednih loginov z napačnim geslom mora vrniti 429 po 5. poskusu.

Mongo test: Pozeni docker compose up. Preveri, da se brez gesla ne moreš povezati na MongoDB.

7.3 Spremljanje po uvedbi

Po uvedbi popravkov v produkciji vzpostavi naslednje spremljanje, da zagotoviš, da so popravki učinkoviti in da ni novih težav:

Beleženje vseh neuspešnih prijav (log s IP, uporabniškim imenom, geslom) in opozorilo po 10 neuspešnih poskusih v 5 minutah.

Beleženje vseh klicov na /api/setup/* endpoint — če se klici pojavljajo po končanem setup-u, je to sumljivo.

Beleženje vseh sprememb config.yaml (file integrity monitoring).

Dnevno preverjanje veljavnosti SSL certifikata (Let's Express avtomatsko obnavlja, vendar lahko odpove).

Tedenski pregled MongoDB povezav (mongo log) za nenavadne IP-je.

Mesečni pregled odvisnosti na znane ranljivosti z Dependabot ali Snyk.

8. Sklep

NutrixPOS je ometajoč odprtokodni POS sistem z aktivnim razvojem, vendar v trenutni obliki še ni primeren za produkcijsko uporabo. Najdene kritične ranljivosti omogočajo popoln kompromis sistema v kratkem času in z minimalnim naporom napadalca. Vse tri kritične ugotovitve so posledica primanjkljajev v osnovni varnostni konfiguraciji in ne zahtevajo prestrukturiranja aplikacije — z pripravljenimi popravki jih je mogoče odpraviti v 1–2 delovnih dneh.

Po aplikaciji vseh pripravljenih popravkov in naslovitvi visokih ugotovitev (HTTPS, httpOnly cookie, audit log) lahko projekt doseže raven varnostne zrelosti, ki je primerna za majhne notranje namestitve (enota prodajna mesto, zaupno omrežje). Za večje namestitve ali namestitve z javno izpostavljenostjo je priporočljiv dodaten formalni pregled s strani certificiranega revizorja, predvsem če bo sistem procesiral plačilne kartice (PCI-DSS obvezen).

Priporočamo tudi, da projekt vzpostavi proces za sprotno nasavljanje varnostnih težav: (1) CI pipeline z golangci-lint in eslint, (2) Dependabot za avtomatsko posodabljanje odvisnosti, (3) vsaj osnovne enote teste za avtentikacijo in avtorizacijo, (4) letni varnostni pregled s strani zunanega revizorja. Te investicije se bodo obrestovale, ker bodo preprečile, da bi se podobne težave ponovile v prihodnosti.

Končna ocena varnostne zrelosti

Kategorija	Trenutno	Po popravkih	Cilj (12 mesecev)
Avtentikacija	2/5	4/5	5/5
Avtorizacija	3/5	4/5	5/5
Konfiguracijska varnost	1/5	4/5	5/5
Mrežna varnost	1/5	4/5	5/5
Logiranje in nadzor	1/5	2/5	4/5
Varnost v CI/CD	1/5	2/5	4/5
Skupna ocena	1.5/5	3.3/5	4.5/5

Ciljna ocena 4.5/5 po 12 mesecih je dosegljiva z aplikacijo vseh popravkov iz tega pregleda in z uvajanjem osnovnih varnostnih praks (testi, CI linting, audit log, formalni letni pregled).